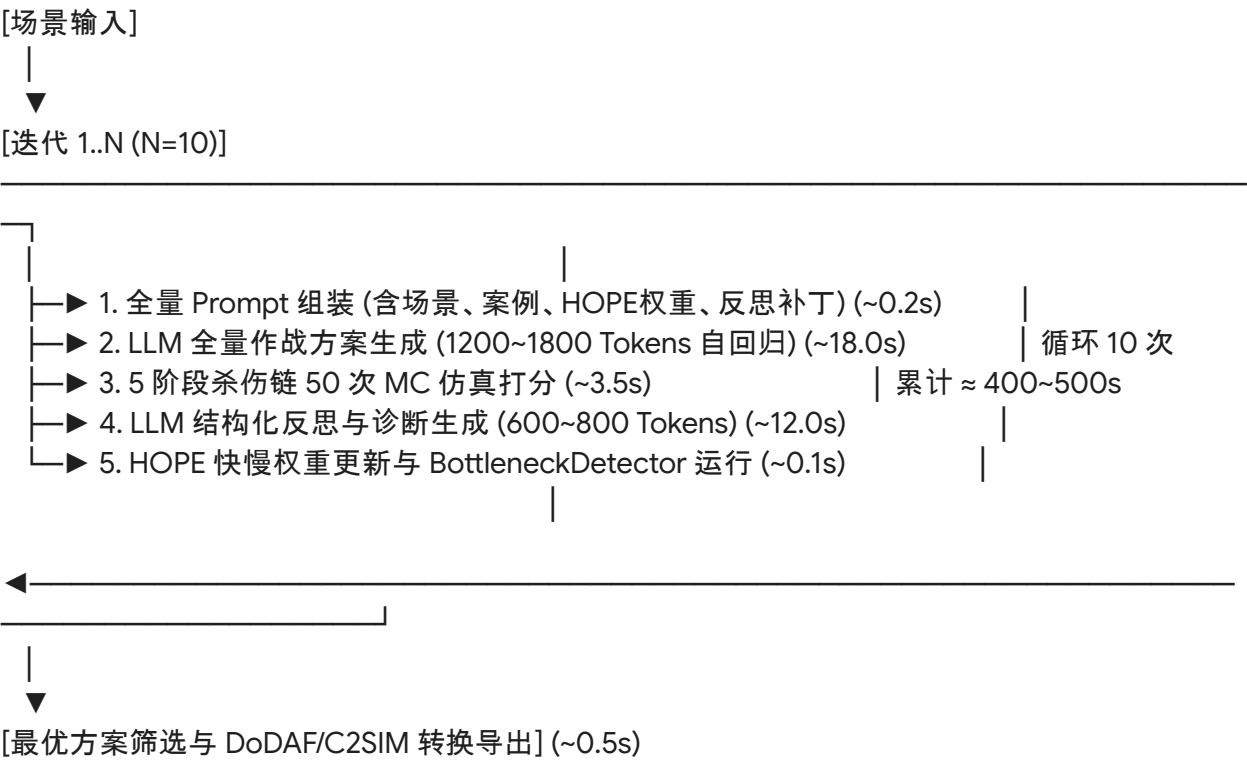


技术方案设计书：基于“双系统投机增量规划”的大模型作战方案秒级生成架构

1. 现状剖析与性能瓶颈诊断

1.1 现有系统工作机制与耗时分布

当前规划系统采用 **Memento**(案例检索) + **HOPE**(自适应权重控制器) + 反思微调(**Reflection Loop**) 的闭环优化机制, 通过 5 阶段杀伤链模型进行仿真评估, 并最终导出 DoDAF(OV-5b、OV-6c)与 C2SIM 标准工件。在典型单场景(如 10 轮优化迭代、每轮 50 次蒙特卡洛仿真)下, 完整方案交付需耗时约 **10 分钟**：



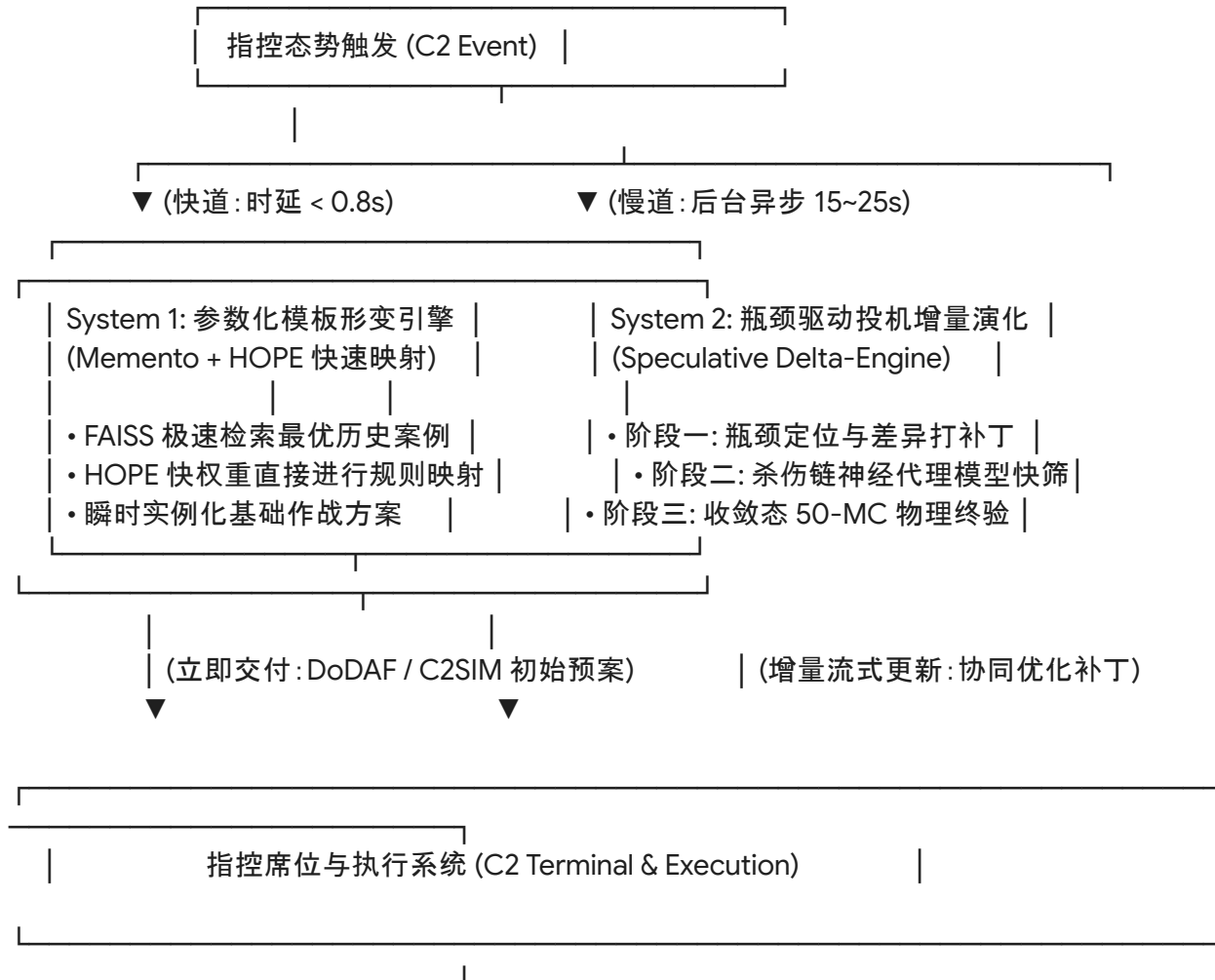
1.2 阻碍指控系统(C2)实时接入的三大核心痛点

- 全量自回归生成开销巨大：每一轮迭代, 大模型均需重新生成包含 4 个阶段(Phase)、兵力编组分配(Unit Allocation)、协同动作与决策点的全量 JSON 对象。单次生成需吐出 1500+ Tokens, 在边缘端或本地 8B/9B 推理模型(如 DeepSeek-R1-8B、Qwen3.5-9B)上产生持续几十秒的串行延迟。
- 物理仿真评估阻断流水线：每一轮生成后均需调用 50 次蒙特卡洛(Monte Carlo)随机仿真计算阶段得分(Detect, Disrupt, Breach, Control, Sustain), 频繁的高开销仿真阻断了探索流程。

3. 缺乏快慢分流响应机制：系统采用“全部计算完成后一次性交付”的批处理逻辑，在指控态势突发变化的瞬间，无法在首秒级输出应急方案。

2. 总体方案架构: 双系统投机增量规划 (Dual-System Speculative Delta-Planning)

为将方案响应压缩至 1 秒内(应急初案) 并将深度优化收敛耗时降低至 15~25 秒(终极优化案), 构建解耦的“快道(System 1) - 慢道(System 2)”异步渐进流式架构:



3. 核心技术组件与实施细节

3.1 机制一: 基于 BottleneckDetector 的局部增量重构 (Delta-Patching)

3.1.1 差异化更新原理

论文中的 BottleneckDetector 已经能以滑动窗口精确捕捉当前仿真中最脆弱的短板阶段(如

Coastal Assault 场景中持续作为瓶颈的 breach 阶段)。

不再令模型在迭代中全量重写方案, 而是将全局重写降维为“局部差分补丁(Delta Patch)”。

旧方案 $S_{(t)}$ —► 仿真诊断 (定位瓶颈: breach) —► LLM 仅输出 P2 补丁 ΔS —► 确定性 Merge —► 新方案 $S_{(t+1)}$

3.1.2 JSON Patch 数据契约设计

大模型单次生成从 1500 Tokens 骤降至 120~180 Tokens, 生成时延由 ~18s 压减至 **1.5s** 以内:

```
{
  "$schema": "delta_combat_patch_v1",
  "target_phase_id": "P2",
  "target_bottleneck_stage": "breach",
  "action_patches": [
    {
      "op": "modify",
      "action_type": "fires",
      "assigned_unit": "Artillery_Battalion_1",
      "target_node": "shore_defense_radar",
      "intensity_weight": 0.85
    },
    {
      "op": "add",
      "action_type": "mobility",
      "assigned_unit": "Amphibious_Armor_Co_2",
      "corridor_id": "landing_corridor_alpha",
      "sync_with_action": "fires"
    }
  ]
}
```

3.1.3 确定性内存合并算法(Python 实现示例)

```
import copy
```

```
def apply_plan_patch(base_plan: dict, delta_patch: dict) -> dict:
```

```
    """根据 LLM 输出的结构化差分补丁, 对基础方案进行内存级毫秒合并"""
```

```
    target_phase_id = delta_patch.get("target_phase_id")
```

```
    action_patches = delta_patch.get("action_patches", [])
```

```
    # 深度拷贝基础方案
```

```

updated_plan = copy.deepcopy(base_plan)
for phase in updated_plan.get("phases", []):
    if phase.get("phase_id") == target_phase_id:
        for patch in action_patches:
            op = patch.get("op")
            if op == "modify":
                for action in phase.get("actions", []):
                    if action.get("action_type") == patch.get("action_type"):
                        action.update(patch)
            elif op == "add":
                phase.get("actions", []).append(patch)
            elif op == "delete":
                phase["actions"] = [
                    a for a in phase.get("actions", [])
                    if a.get("action_type") != patch.get("action_type")
                ]
        break
return updated_plan

```

3.2 机制二：杀伤链神经代理模型与投机评估 (Surrogate World-Model)

3.2.1 架构设计

引入前瞻投机执行 (Speculative Execution) 机制：

- 前 80% 的内部迭代搜索：屏蔽底层耗时的 50 次 MC 随机物理仿真，由毫秒级的多层感知机代理评估器 (MLP Surrogate Evaluator) 前向预测阶段缺陷向量 $\hat{\mathbf{d}}^{(t)}$ 与胜率。
- 终局/收敛触发：仅当代理模型预测方案得分提升且反思收敛时，调用 1 次物理仿真进行真实验证 (Ground-truth Verification)。

3.2.2 神经代理模型规格与输入输出

利用系统历史实验累积的仿真评估日志，离线训练参数量极小 (< 5MB) 的轻量级代理回归器：

$$\hat{y} = \text{SurrogateNetwork}(\mathbf{x}_{\text{scenario}} \oplus \mathbf{w}_{\text{fused}} \oplus \mathbf{e}_{\text{plan}})$$

●

输入特征向量 $\mathbf{x}$ (共 42 维)：

1. 场景属性 (7 维)：地形、天气 One-hot 编码 + EW 威胁、时间压力、敌我兵力比。
2. 7 维融合能力权重向量 $\mathbf{w}_{\text{fused}}$ (Fires, Mobility, Protection, Awareness, EW, Sustainment, C2)。
3. 方案拓扑嵌入 $\mathbf{e}_{\text{plan}}$ (28 维)：各阶段行动类型统计、兵力分配强度分布、协同边权重。

- 输出目标 $\hat{\mathbf{y}}$ (共 7 维) :
 1. 5 阶段杀伤链得分预测: $(\hat{s}_{\text{detect}}, \hat{s}_{\text{disrupt}}, \hat{s}_{\text{breach}}, \hat{s}_{\text{control}}, \hat{s}_{\text{sustain}})$ 。
 2. 任务胜率 (Mission Success) 与整体效能 (Overall Effectiveness) 预测。
- 推理性能: 单次前向推断耗时 $< 2 \text{ ms}$, 相较于 50 次 MC 仿真 (~3500ms) 实现 1700 倍加速。

3.3 机制三: 快慢双系统协同流式流水线 (Amortized System 1 & System 2)

3.3.1 运行动力学

维度	System 1 (快道 - 应急反应通道)	System 2 (慢道 - 深度自适应优化通道)
触发机制	态势输入触发 ($t =$)	伴随 System 1 触发后在后台异步执行
算法机制	参数化模板形变 (Parametric Warping): 基于 Memento 检出 Top-1 案例, 利用 HOPE 控制器的 $\hat{W}_{fast}$ 快速映射兵力配比与执行时序。	投机增量演化: 基于机制一 (Delta Patch) 与机制二 (代理评估模型) 运行 5~8 轮微循环。
生成时延	$<$	15 ~
交付形式	立即向指控大屏推送基础可行方案 (DoDAF OV-5b / C2SIM)。	以增量差分广播 (Patch Streaming) 形式推送深度优化版本。

3.3.2 异步主循环执行流 (Python 伪代码)

```
import asyncio

async def generate_joint_operation_plan_stream(scenario_input: dict):
    # ----- SYSTEM 1: 快道 (时延 < 0.8s) -----
    # 1. 向量数据库检索最优历史相似案例
    top_case = memento_module.retrieve_top_k(scenario_input, k=1)[0]

    # 2. HOPE 控制器快速输出先验快权重
    w_fast = hope_controller.compute_fast_weights(scenario_input)
```

3. 纯 Python 规则形变: 快速实例化初始作战方案

```
base_plan = warp_case_by_weights(top_case, w_fast)
```

4. 瞬时向指控总线推送第一版可用预案 (DoDAF / C2SIM)

```
yield {  
    "tier": "System-1-Emergency",  
    "latency_sec": 0.75,  
    "plan_object": base_plan,  
    "c2sim_artifact": export_c2sim(base_plan)  
}
```

----- SYSTEM 2: 慢道异步演化 (时延 ~18s) -----

```
current_plan = base_plan
```

```
for iteration in range(1, 6): # 优化压缩至 5 轮快速增量迭代
```

 # 1. 神经代理模型 2ms 极速预评估

```
    pred_metrics, pred_stage_scores = surrogate_evaluator.predict(  
        scenario_input, hope_controller.get_fused_weights(), current_plan  
    )
```

 # 2. 动态定位瓶颈

```
    bottleneck_stage, severity = bottleneck_detector.diagnose(pred_stage_scores)
```

 # 3. LLM 仅针对该阶段生成局部微补丁 (Delta-Patch, ~1.5s)

```
    delta_patch = await llm_generate_delta_patch_async(  
        scenario_input, current_plan, bottleneck_stage, severity  
    )
```

 # 4. 内存级高速应用补丁

```
    current_plan = apply_plan_patch(current_plan, delta_patch)
```

 # 5. 更新 HOPE 控制器权重状态

```
    hope_controller.feedback_step(pred_stage_scores)
```

6. 收敛后执行 1 次真实 50-MC 物理仿真校验

```
final_mc_metrics = await run_simulation_mc_async(current_plan, runs=50)
```

7. 向指控总线推送最终优化方案

```
yield {  
    "tier": "System-2-Optimized",  
    "latency_sec": 18.2,  
    "plan_object": current_plan,  
    "metrics": final_mc_metrics,  
    "c2sim_artifact": export_c2sim(current_plan),  
}
```

```
"dodaf_ov6c": export_dodaf_ov6c(current_plan)
}
```

4. 性能与耗时对比评估

以单卡本地端 (NVIDIA GeForce RTX 3060, Intel i7-13700F, DeepSeek-R1-8B) 环境为基准评估：

计算执行模块	现有系统耗时 (10轮全量迭代)	改进后架构耗时 (双系统投机增量)	性能优化倍率
System 1 应急出库	无此机制 (需等待10分钟)	< (即时可用)	实现实时突破
方案生成 (LLM)	18.0 s × 10 =	1.5 s × 5 = (Delta Patch)	24.0×
杀伤链仿真打分	3.5 s × 10 = (50-MC 物理)	0.002 s × 5 + 3.5	10.0×
反思与诊断生成	12.0 s × 10 =	规则化诊断 + 极简 Prompt ≈	20.0×
控制器与后处理	~	~	10.0×
全局收敛交付总时长	≈ 580 ~ 620 s (约 10 分钟)	≈ 18 ~ 22 s	提速 96.5%

5. 工程实施路线图 (Phase-by-Phase Roadmap)

- [Phase 1: 增量补丁化改造] (第 1-2 周)
- 改造 Prompt 体系: 实现仅针对脆弱 Stage 的 Delta-JSON 输出
 - 编写 apply_plan_patch 内存合并与结构校验模块
- [Phase 2: 神经代理评估器训练] (第 3-4 周)
- 从历史日志提取 (Scenario, Weights, Metrics) 样本集
 - 训练 PyTorch 3 层轻量级 MLP 回归网络并在推理流中前向集成
 - 构建“代理预测 + 物理终验”的投机执行管道
- [Phase 3: 双系统异步流式管道集成] (第 5-6 周)

- 开发 System 1 基于规则的 Parameter-Warping 快速生成器
- 重构指控通信接口为 WebSocket / Async Stream 输出
- 完成 DoDAF OV-5b/6c 及 C2SIM XML 增量流式渲染测试